SDG BLOCKCHAIN ACCELERATOR

Debugging & Testing Report

TradeChainx & UNDP Ethiopia

Project Name:
Blockchain-Based Carbon Credits MRV Platform

Challenge & UNDP Office: SDG Blockchain Accelerator

Report Version: v1.0

2. Testing Approach

Unit Testing:
Aiken unit tests were written to validate core contract logic, including project registration, state transitions, verification checks, minting rules, and token retirement. Tests were structured per validator and executed using the `aiken test`.

Integration Testing:
End-to-end flows were tested using the Aiken emulator and Cardano testnet. Lucid was used to construct transactions simulating real user interactions via wallets and the React dashboard.

Edge Cases Tested:

- Invalid datum structure rejected
- Invalid or missing redeemer rejected
- Unauthorized verifier signature rejected
- Mint attempt before verification blocked
- Double minting and double spend attempts blocked
- Token retirement without ownership rejected

Example Entry: Test to check that registry Validation Fails When Not Signed By Developer

```
test test_verify_success_with_developer_signature() {
  let regulator = #"010203"
  let developer = #"040506"
  let datum = ProjectDatum {
    project_id: "test",
    developer_pkh: developer,
    status: PendingVerification,
    metadata_cid: "ipfs_hash_here",
    last_issuance_period: 0,
    total_credits_issued: 0,
  }
  let redeemer = Verify
  let self = Transaction { ..placeholder, extra_signatories: [developer] }
  let utxo = OutputReference { transaction_id: "", output_index: 0 }

  registry.spend(regulator, Some(datum), redeemer, utxo, self)
}
```

```
1 tests | 1 passed | 0 failed

registry
PASS [mem:  28.65 K, cpu:  10.32 M] test_verify_success
PASS [mem:  28.65 K, cpu:  10.32 M] test_verify_success_with_developer_signature
PASS [mem:  29.25 K, cpu:  10.49 M] test_verify_fail_wrong_status
PASS [mem:  36.44 K, cpu:  12.99 M] test_issue_success
PASS [mem:  33.55 K, cpu:  10.57 M] test_decommission_by_developer
5 tests | 5 passed | 0 failed
```

3. Error Logs

Example of a failed unit test for registry validation when the project status is in the wrong state

```
registry
PASS [mem:  28.65 K, cpu:  10.32 M] test_verify_success
PASS [mem:  28.65 K, cpu:  10.32 M] test_verify_success_with_developer_signature
FAIL [mem:  28.65 K, cpu:  10.32 M] test_verify_fail_wrong_status
PASS [mem:  36.44 K, cpu:  12.99 M] test_issue_success
PASS [mem:  33.55 K, cpu:  10.57 M] test_decommission_by_developer
5 tests | 4 passed | 1 failed
```

4. Resolved Issues

Moved Wallet to Backend.

5. Optimization Notes

Updates made to the registry validator for optimization:

Reordered condition checks	Cheapest checks (ID match, hash validation) evaluated first → fail-fast on invalid inputs
Consolidated signature checks	Replaced two separate list.has calls with single list.all iteration
Used pipe syntax for helper function	More readable and eliminates intermediate variable
Simplified fold operation	Removed unused _name parameter and used cleaner pipeline

6. Tools and Environments Used

- Aiken CLI (build, check, test, emu)
- Cardano-node (Testnet / Preprod)
- Lucid (transaction construction and submission)
- Supabase (off-chain MRV data storage)
- React (user dashboard)
- Local emulator for dry-run testing

- Wallets: Nami, Lace

7. Remaining Issues / Next Steps

- Further reduce validator execution units for large-scale minting
- Expand test coverage for extreme edge cases
- Run stress tests on Cardano Preprod
- Prepare contracts for third-party security audit
- Add monitoring for production deployment